6.7 Trace and verify the application

Use the completed application from 6.6. This tutorial adds no files. Trace the program and verify that each responsibility has a clear home.

Follow a Products request

1. The browser requests `/tutorials/class6/index.php?action=products`.
2. index.php loads the existing Composer autoloader, creates Application, and calls run.
3. run constructs the page controllers. DefaultController inherits Controller's constructor; ProductController calls it explicitly before initializing its products property.
4. Application reads action and the products case calls ProductController::productList.
5. productList passes the products property in the products context entry to Twig. Product's hooks validated the objects when the constructor created them.
6. Twig renders products.html.twig with the layout from base.html.twig. The controller echoes the returned HTML string.
7. The browser displays the HTML and requests style.css through its relative URL.

The request passes through the entry script, front controller, page controller, and view. Controller is involved through inherited initialization; it is not another route or an object we construct directly.

Follow a single-product request

Click the Desk organizer link. The browser requests `?action=product&id=2`. Application validates the ID as a positive integer, then calls showProduct(2). ProductController retrieves key 2 from its products property and passes that object under the singular product context key. Twig renders product.html.twig with the shared layout. A malformed ID is rejected by Application; a valid integer absent from the collection is rejected by ProductController. Both paths use productNotFound to return a 404 view.

Match responsibilities to files

Responsibility	**Owner**
Load Composer and start the application	public/tutorials/class6/index.php
Read action, validate the detail ID, and choose a known page method	Application
Initialize Twig for each page controller	Abstract Controller
Prepare Home or Contact	DefaultController
Initialize the product collection, look up an item, and render list/detail/error pages	ProductController
Represent and validate one product's name and price	Product
Define shared markup and page-specific presentation	Base and child Twig templates

Complete the browser checkpoint

Request suffix	Expected result
No query string	Home
`?action=home`	Home
`?action=contact`	Contact
`?action=products`	Products and three name/price pairs
`?action=product&id=2`	Desk organizer, $12.00, and a back link
`?action=product`	Product not found, HTTP 404
`?action=product&id[]=2`	Product not found, HTTP 404
`?action=product&id=abc`	Product not found, HTTP 404
`?action=product&id=999`	Product not found, HTTP 404
`?action=unknown`	Home fallback
`?action[]=products`	Home fallback without a type warning

On each page, check its tab title, heading, shared navigation, and active link. Detail and product-error pages keep Products active. Confirm the response contains one document rather than concatenated pages. Repeat the temporary empty-array check in the ProductController constructor, then restore the product data.

Rehearse adding a feature

For another basic page, plan a public DefaultController method, a child template, an explicit Application case, and a link in base. For another family of pages, plan a specialized Controller subclass and the data class it needs. The reinforcement exercises put both approaches into practice.

Ask students to explain these mistakes before fixing them: putting the switch in a Twig template, creating Product objects in Application, making a page method private, or copying Twig initialization into every new subclass. Each either mixes responsibilities or breaks the intended access and inheritance rules.

Finish by opening the reinforcement exercises. Build those in the separate exercise namespace and folders so both sites can run independently.